# Determinants of the Shared/Routed Split in Small Mixture-of-Experts Models

Raphaël Couturier, Stéphane Cuenat
Université Marie et Louis Pasteur, CNRS,
Institut FEMTO-ST (UMR 6174), Belfort, France
{raphael.couturier, stephane.cuenat}@umlp.fr

Pablo Gregori
Departamento de Matemáticas, IMAC,
Universidad Jaume I de Castellón,
Campus Riu Sec s/n,
E-12075 Castellón de la Plana, Spain
gregori@uji.es

***Abstract*—Small Mixture-of-Experts language models built with an always-on shared expert appear to under-use their routed experts: removing the entire routed branch costs little in perplexity, while removing the single shared expert costs a great deal. We show that this asymmetry reflects optimization dynamics and architectural proportion, not the computation being performed. Across models of 58M to 436M parameters the imbalance is transient: the 134M reference model equalizes its two branches by roughly 300M tokens, while a twice-deeper model stays shared-dominated longer. The cause is a gradient-exposure race that the shared expert, updated on every token, wins against routed experts updated on a $k/E$ fraction of them. The race is controllable: slowing the shared expert reallocates computation to the routed branch at no cost in perplexity, and the reallocation persists even when the handicap covers only the first tenth of training. Where the branches settle is governed jointly by their relative width and by routing granularity, not by the number of experts. At fixed routing an eightfold change in the shared expert's width moves its share of ablation impact from 0.28 to 0.81 at essentially unchanged perplexity; at identical widths, top-1 and top-4 routing land at 0.97 and 0.17.**

***Index Terms*—mixture of experts, training dynamics, sparse models, language models, expert specialization, domain-labelled corpora, data-centric model design**

## I. Introduction

Mixture-of-Experts (MoE) layers replace a dense feed-forward block with a router that dispatches each token to a small number of experts. The DeepSeekMoE design [1] adds an always-on *shared* expert beside the *routed* ones, on the intuition that it absorbs computation common to all tokens and thereby frees the routed experts to specialize. Small MoEs are inexpensive to train and every token can be labelled, which has made them a common testbed for studying how experts divide work.

In this regime a consistent observation recurs. Ablating the entire routed branch barely moves held-out perplexity, whereas ablating the lone shared expert degrades it severely. Taken at face value this indicates that the routed experts are nearly redundant, and it admits two explanations that both attribute the effect to the architecture: over-parametrization of the routed branch, or suppression by the shared expert.

Neither explanation is supported by our measurements, and the imbalance is largely uninformative about what the computation requires. It is the joint product of two things with no obvious connection to the task: how quickly each branch is trained, and how wide each branch is. The apparent redundancy is transient, disappearing once the model has seen a few hundred million tokens. Its cause is a competition for gradient signal that the shared expert wins by construction, and that a single learning-rate multiplier transfers to the routed branch at no cost in perplexity; applying it over the first tenth of training suffices, and the allocation persists long after the handicap is lifted. Where the branches come to rest is set jointly by their relative width and by the routing granularity, not, as the mechanism naively suggests, by the number of routed experts: holding the routed branch identical down to its parameter shapes and varying only the shared expert's width moves the shared branch's share of ablation impact from 0.28 to 0.81, while at identical widths top-1 and top-4 routing land at 0.97 and 0.17. Granularity is the only one of these levers with a measurable cost, and single-expert routing is its degenerate limit.

The question is data-centric as much as architectural. What the routed experts are supposed to acquire is structure present in the training corpus, and we measure that directly: every token carries the label of the corpus it was drawn from, so expert assignment can be scored against data domain rather than inferred from inspection. The diagnostic error we describe is a measurement error about data-driven structure, made at the wrong point in training.

Two senses of redundancy must be distinguished: a branch is ablation-redundant if removing it is cheap, functionally redundant if the other could have carried its work. Our results separate them. With enough training the routed experts are not ablation-redundant at all, while the branches are close to interchangeable in where the computation may reside. The split is therefore a largely free design choice: among configurations of comparable size, perplexity varies by less than 0.1 while the branch split spans 0.25 to 0.81. This freedom is relevant whenever the routed experts are themselves the object of study, as in interpretability, expert pruning and modular editing [2], [3], and it means that a diagnosis of expert redundancy drawn from an undertrained checkpoint is misleading rather than merely imprecise. The choice of lever then matters: widening the routed branch relative to the shared one more than doubles domain specialization, whereas the

learning-rate control does not reliably move it.

## II. Related Work

Conditional computation through a sparsely gated MoE layer was introduced by Shazeer et al. [4] and scaled by GShard [5] and Switch Transformer [6], which popularized the auxiliary load-balancing loss in the form we use; ST-MoE [7] added the router $z$-loss for stability. Our reference configuration is top-2, in the style of Mixtral [8].

The always-on shared expert originates with DeepSeek-MoE [1] and carries through DeepSeek-V2 and V3 [9], [10] and open families such as Qwen2 [11], on the argument that isolating common knowledge in a shared branch lets the routed experts specialize more sharply. DeepSeekMoE pairs it with a second argument, that finer expert segmentation sharpens specialization, of which Section VI-A tests a controlled analogue in isolation. Both concern what the branches represent; our results concern how they are trained and proportioned, and suggest that at small scale the latter dominates. Observational studies report that experts organize more by token identity and surface form than by topic [2], [8], and Wang et al. [12] argue that routing reflects hidden-state geometry rather than domain expertise; these characterize training's outcome, while we add a causal probe and a dynamical explanation. Mouzouni [13] finds, across the checkpoints of large open MoEs, that routing passes through distinct phases, so that what an early checkpoint shows about load balance is not what training ends with; we reach the parallel conclusion for branch importance. Our mechanism instantiates a broader phenomenon, the early phase of training committing a network to structure that later training refines rather than revisits [14], [15], a phase whose routing instability motivated StableMoE [16] and whose expert divergence has recent theory [17].

## III. Experimental Setup

*a) Model:* A decoder-only MoE in native PyTorch, built on the open-source Tiny-MoE implementation [18], with Multi-head Latent Attention [9], RoPE [19] and RMSNorm [20]. Each layer carries $E$ routed experts and one shared expert with top-$k$ routing. The reference configuration has $E = 8$, $k = 2$, 8 layers, hidden size 512, context 512 and 134M parameters; every expert, routed or shared, has intermediate width 1024. Section VI varies the two widths independently, writing them $I$ per routed expert and $I_{\text{shared}}$ for the shared one; configurations range from 58M to 436M parameters. All runs use AdamW ($\beta_1$=0.9, $\beta_2$=0.95, decay 0.1), cosine to zero after 1500 warmup steps, batch 64 at length 512, peak learning rate (LR) $3 \times 10^{-4}$, load-balancing coefficient $10^{-2}$, router $z$-loss $10^{-3}$, bfloat16, capacity factor 1.25 in training and none in analysis; 30 000 steps is 983M tokens, on one RTX A6000.

*b) Data:* Pre-training streams three token-labelled corpora: FineWeb-Edu [21] (60%), Cosmopedia-v2 [22] (25%) and OpenWebMath [23] (15%), tokenized with a 32k vocabulary and packed to 512 tokens. Held-out evaluation uses a skip-offset of the same streams rather than a disjoint split, so absolute perplexities may be optimistic through near-duplication; every claim below rests on differences between conditions evaluated on the identical stream, not on levels.

*c) Measuring where the computation lives:* Perplexity (PPL) is the exponential of the mean token-level cross-entropy on the held-out stream, $\mathrm{PPL} = \exp(-N^{-1} \sum_t \log p(x_t \mid x_{<t}))$, so at the values reported here a difference of 0.1 PPL is about 0.03 nats per token. We ablate a whole branch, either all routed experts or the shared expert, and record the resulting increase in PPL; single-expert ablation is uninformative under top-$k$ routing, since the router reroutes to the survivors. We summarize the two measurements by the shared branch's share of total ablation impact, $s = \Delta_{\text{shared}}/(\Delta_{\text{shared}} + \Delta_{\text{routed}})$, so that $s = 0.5$ denotes branches of equal importance and $s \to 1$ a shared-dominated model. Almost every trajectory dips to a minimum and then rises slowly, the exception being the top-1 configurations of Section VI-A, which decline monotonically; we therefore report the minimum reached over the trajectory and, where the endpoint matters, the value at 983M tokens. Every configuration in Tables II, III and IV has at least two seeds, four for the reference, which covers top-1 and both extremes of the width ratio, where the trajectories behave least alike; across them the seed standard deviation on $s$ is at most 0.03, and under 0.01 at top-1, an order of magnitude below every effect discussed here. Single-seed runs are identified where they appear. We report perplexity and ablation impact rather than downstream accuracy because at this scale standard benchmarks sit near chance and would not resolve the differences studied here; the question is where the computation lives, not what it is worth on a task. As a reference point, a dense feed-forward network at the reference model's active width (58M parameters) reaches perplexity 3.11 at 983M tokens against the reference MoE's 3.00, and a variant trained without the shared expert reaches 3.03: the architecture justifies its inactive parameters, while the always-on branch contributes roughly in proportion to its parameter share, its distinctive contribution being organizational (Section VII).

*d) Whether $s$ measures computation or output norm:* Zero-ablation perturbs the residual stream by an amount that scales with the removed branch's output norm, and both levers studied below modulate that norm. Table I validates the statistic. Substituting each ablated branch's per-layer mean output for zeros reproduces $s$ within 0.07 everywhere, ordering unchanged. The norm share dissociates from $s$ in both directions: under top-1 the routed branch holds 24% of the norm and 3% of the impact, and under a slowed shared expert the shared branch holds 32% of the norm and 10% of the impact. A fixed scalar of 2 or 0.5 on the shared output moves the split by at most 0.07 at flat perplexity, the network *absorbing* it by rescaling weights until the trained norm share returns within 0.01 of baseline, whereas width manipulations, which no rescaling absorbs, move $s$ by 0.32. Contribution norm follows function here; it does not set it.

TABLE I
MEASUREMENT VALIDITY OF THE SPLIT STATISTIC

| configuration | $s_0$ | $s_\mu$ | $\nu$ |
|---|---|---|---|
| shared LR $\times 0.1$ | 0.10 | 0.11 | 0.32 |
| top-4 | 0.19 | 0.21 | 0.44 |
| $I_{\text{shared}}$=512 | 0.34 | 0.35 | 0.45 |
| $I$=2048 | 0.40 | 0.43 | 0.48 |
| scalar $\times 2$ | 0.43 | 0.48 | 0.52 |
| scalar $\times 0.5$ | 0.46 | 0.53 | 0.50 |
| reference | 0.50 | 0.54 | 0.51 |
| shared LR $\times 0.3$ | 0.52 | 0.55 | 0.45 |
| $I$=256 | 0.80 | 0.81 | 0.61 |
| $I_{\text{shared}}$=4096 | 0.82 | 0.86 | 0.67 |
| top-1 | 0.97 | 0.97 | 0.76 |

Zero-ablation $s_0$, mean-ablation $s_\mu$ and norm share $\nu$, measured at 983M tokens; the two learning-rate rows at 491M.

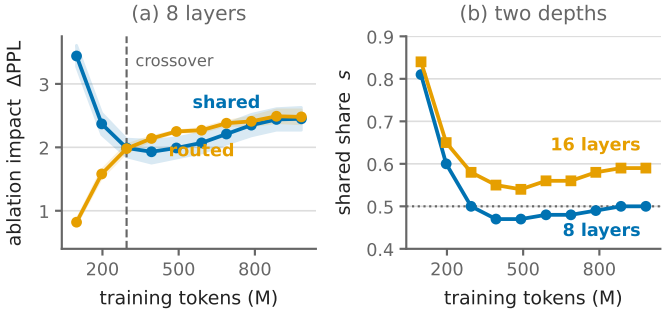



Fig. 1. (a) Ablation impact of removing the shared expert against all eight routed experts, four seeds, bands $\pm 1$ standard deviation. (b) The same measurement as the share $s$, at two depths: the 8-layer model crosses to equal importance, the 16-layer model does not within 983M tokens.

## IV. TOKEN-DEPENDENCE OF BRANCH IMPORTANCE

We train the reference model for 983M tokens with four seeds and measure branch importance at every checkpoint (Fig. 1a). At 98M tokens removing the shared expert costs $+3.44 \pm 0.17$ perplexity against $+0.82 \pm 0.03$ for the whole routed branch, so the shared expert accounts for 81% of the ablation impact and the routed experts appear close to redundant. This is the regime in which many pilot studies operate, including our own earlier experiments. By 295M tokens the branches have equalized ($+1.99$ against $+1.98$, $s = 0.50$) and they remain so through 983M ($+2.45 \pm 0.18$ against $+2.48 \pm 0.11$), while perplexity falls from 4.18 to 3.00. Routed specialization grows in parallel, the expert–domain normalized mutual information (NMI, defined in Section VII) rising from 0.0008 at initialization to 0.0174.

Depth delays the crossing, in the direction that makes the phenomenon harder to detect rather than easier (Fig. 1b). A 16-layer, 251M-parameter model never crosses within our budget: it reaches a minimum of $s = 0.54$ near 491M tokens and drifts back to 0.59 by 983M. That baseline is a single run, though the sweep replicate of Section V tracks it to within 0.02 over their common range. More residual capacity allows the shared expert's initial advantage to compound further, so the larger the model, the longer an observer may incorrectly conclude that its routed experts are redundant.

The implication is narrow but robust. Branch importance is not a property of the architecture that can be measured once; it is a function of how much data the model has seen. Conclusions about routed redundancy drawn below a few hundred million tokens do not survive further training, and the effect is large enough to change the sign of a comparison.

TABLE II
SHARED-EXPERT LEARNING-RATE MULTIPLIER

| $m$ | remove shared | remove routed | $s$ | PPL |
|---|---|---|---|---|
| 1.0 (baseline) | $+3.46 \pm 0.12$ | $+0.85 \pm 0.01$ | 0.80 | 4.16 |
| 0.3 | $+2.48 \pm 0.02$ | $+1.94 \pm 0.00$ | 0.56 | 4.10 |
| 0.1 | $+0.72 \pm 0.08$ | $+4.46 \pm 0.11$ | 0.14 | 4.10 |

Two seeds per condition, measured at 98M tokens. Ablation impacts are increases in held-out PPL.

## V. MECHANISM: A GRADIENT-EXPOSURE RACE

The asymmetry originates in the update rule rather than in the trained weights. Writing the block output on token $t$ as $y_t = f_{\text{sh}}(x_t) + \sum_{e \in \text{topk}(t)} g_{t,e}\, f_e(x_t)$, with $f_{\text{sh}}$ the shared expert, $f_e$ the routed ones and $g_{t,e}$ the router's gate weight, a batch of $N$ tokens delivers gradient from all $N$ of them to the shared parameters but from only about $Nk/E$ to any given routed expert. Under a common learning rate the shared expert therefore accumulates roughly $E/k$ times more effective updates per parameter during the early phase, when gradients are large and the allocation of work is still plastic. The residual stream compounds this effect by routing common computation to whichever submodule reduces the loss fastest. The account predicts that slowing the shared expert should let the routed experts capture computation earlier, and that the crossover should scale with $E/k$. The first holds; Section VI is devoted to what replaces the second.

### A. Learning-rate handicap on the shared expert

We place the shared expert's parameters in their own optimizer group at $\text{lr} \times m$ and leave everything else untouched. Table II reports the state at 98M tokens, deep in the regime where the baseline looks shared-dominated. Slowing the shared expert makes the routed branch load-bearing far earlier: a fivefold increase in routed ablation impact at a fixed token budget, shifting the crossover from roughly 295M tokens to below 200M at $m = 0.3$ and to before training settles at $m = 0.1$ (Fig. 2a). The effect is monotone and dose-dependent, and it refutes the hypothesis that routed experts mature on an intrinsic schedule: their updates are dominated by those of the shared expert.

These runs use a 15 000-step schedule; at 98M tokens, where Table II is read, the schedules' learning rates differ by under three percent, so the contrast is the multiplier. (The baseline row is the reference run over two seeds, hence its last-digit difference from the four-seed reading above.) Perplexity is unchanged across conditions and marginally lower when the shared expert is slowed, so the shared expert is not necessary

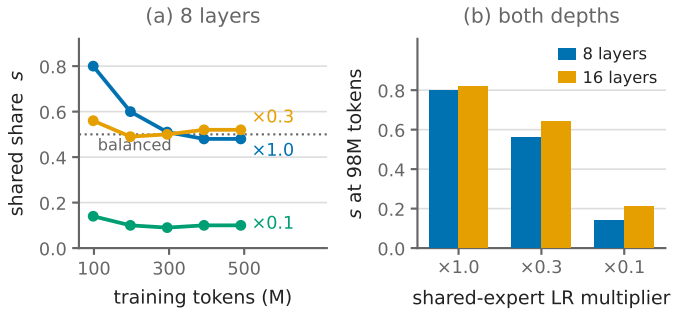



Fig. 2. (a) Shared branch share against training tokens for three shared-expert learning-rate multipliers, two seeds. (b) The same control read at 98M tokens, at both depths.

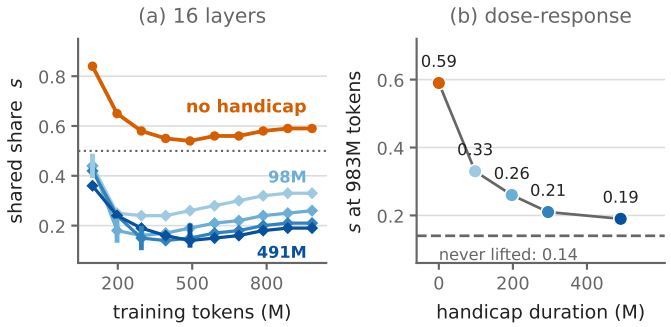



Fig. 3. (a) Shared branch share at 16 layers for four handicap durations, the tick marking where each handicap was lifted. (b) The resulting dose-response, with the never-lifted limit dashed. Every ramped condition ends within 0.01 perplexity of the others.

for the computation it performs in the baseline. The allocation reflects the rate at which each branch learns, not a requirement of the computation.

Interpolating Table II in $\log m$, the branches would balance at 98M tokens near $m \approx 0.26$, and near 0.21 in the 16-layer sweep below, close to the $k/E = 0.25$ at which per-parameter exposure is equal. We note the coincidence without leaning on it: it is a two-point interpolation at a single $k/E$, it holds at one budget only, since by 295M tokens the unhandicapped model balances by itself, and the response is not monotone in exposure. Accelerating the shared expert instead, at $m = 3$ over the full schedule, also lowers $s$, to **??** at 983M tokens with perplexity **??** (two seeds), so the race can be lost by converging too early as well as by learning too slowly. The handicap is the direction we characterize.

A mechanism specific to one depth would be of limited interest, so we repeat the sweep at 16 layers, where it reproduces almost exactly (Fig. 2b): the shared share at 98M tokens falls from 0.82 to 0.64 to 0.21 across the same three multipliers, against 0.80, 0.56 and 0.14 at 8 layers, and perplexity is again unchanged. The deeper baseline reads 0.82 in this sweep against 0.84 in the long-schedule run of Section IV, within the seed spread.

### B. Persistence of a temporary handicap

If the split is decided by an early race, a handicap covering only that race should suffice, and the allocation should not revert once it is lifted. We test this at 16 layers by ramping $m$ linearly from 0.1 back to 1.0 by a chosen step, after which every parameter trains at the base rate (Fig. 3).

The reallocation is not reversed. Once the handicap is removed the share stays near where the handicap left it rather than returning to the baseline trajectory, and the shared branch reasserts itself only slowly. The response is monotone in duration: the share at 983M tokens is 0.33, 0.26, 0.21 and 0.19 for handicaps lifted at 98M, 197M, 295M and 491M tokens, against 0.59 with no handicap and 0.14 for a handicap never lifted.

Three features of that curve matter more than its monotonicity. A brief handicap already produces most of the effect: lifting it after 98M tokens, one tenth of the budget, reaches 0.33, more than half of the distance from the baseline to the never-lifted limit. The increments then shrink (0.07, 0.05, 0.02), yet the never-lifted limit sits a further 0.05 below the longest ramp, so the response flattens without plateauing in our range. Releasing the handicap is also marginally preferable to keeping it: final perplexities are 2.88, 2.88, 2.88 and 2.89 for the four ramps, against 2.90 for the baseline and 2.91 when the handicap is never released, the only condition worse than no intervention. The multiplier therefore acts as a schedule intervention rather than a permanent penalty, as a race-then-commit dynamic predicts, since the branch that wins the early competition is the one the residual stream subsequently builds upon.

We replicated the 295M endpoint with a second seed and obtained the same summary statistics, a minimum of 0.14 and 0.21 at 983M tokens. The other four conditions are single runs at one depth and one handicap strength, so we report the shape of the dependence rather than a functional law.

## VI. Determinants of the Resting Split

The mechanism explains why the shared expert leads and shows that the lead can be reversed. It does not say where the split comes to rest, and the natural prediction that the resting point follows $E/k$ is not borne out. We assemble twelve 8-layer configurations at top-2 routing: the six runs of Table III, a capacity-matched sweep over expert count ($E \cdot I = 8192$, $E \in \{4, 8, 16, 32\}$), and three from an earlier ratio-1.0 sweep ($E = 4$, 16, 32 at $I = I_{\text{shared}} = 1024$, 83M–436M parameters; the $E = 4$ run stopped at 491M with its share flat from 295M on, the rest reached 983M).

Table III isolates the two candidates. Both contrasts hold $E = 8$ and $k = 2$ fixed, so expert count, routing and per-expert gradient share are constant throughout. The second is decisive: with $I$ pinned at 1024 the routed branch is identical across the four runs, the same number of experts of the same width receiving the same routed capacity $k \cdot I = 2048$, and yet the split moves by 0.53 (Fig. 4b). Whatever governs the resting point, it is not the absolute capacity of the routed branch.

What the two contrasts share is the ratio $I/I_{\text{shared}}$. Grouping every configuration by that ratio yields bands that do not

TABLE III
BRANCH WIDTH CONTRASTS

| $I$ | $I_{\text{shared}}$ | params | $\Delta_{\text{sh}}$ | $\Delta_{\text{rt}}$ | $s_{\min}$ | $s$ |
|---|---|---|---|---|---|---|
| VARYING ROUTED WIDTH | | | | | | |
| 256 | 1024 | 58.2M | +5.48 | +1.39 | 0.79 | 0.80 |
| 1024 | 1024 | 133.7M | +2.45 | +2.48 | 0.47 | 0.50 |
| 2048 | 1024 | 234.4M | +1.95 | +2.86 | 0.37 | 0.40 |
| VARYING SHARED WIDTH | | | | | | |
| 1024 | 512 | 127.4M | +1.61 | +3.19 | 0.28 | 0.34 |
| 1024 | 1024 | 133.7M | +2.45 | +2.48 | 0.47 | 0.50 |
| 1024 | 2048 | 146.3M | +3.88 | +1.54 | 0.70 | 0.72 |
| 1024 | 4096 | 171.5M | +5.29 | +1.18 | 0.81 | 0.82 |

All runs at $E$=8, $k$=2, two seeds each, four for the reference. Ablation impacts $\Delta$ (increase in held-out PPL) and $s$ at 983M tokens, $s_{\min}$ over the trajectory. In the lower block the routed branch is identical across the four runs.

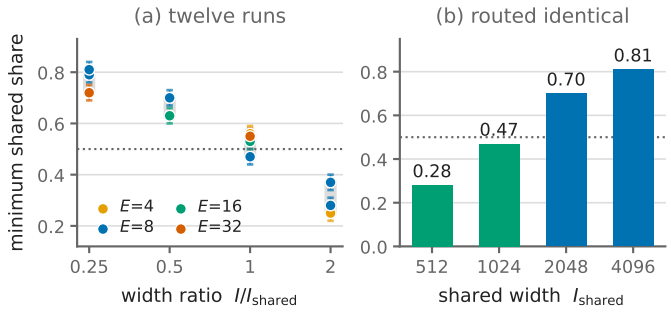



Fig. 4. (a) Minimum shared share against the branch width ratio, one point per configuration, coloured by expert count, with bars of $\pm 0.03$, the largest seed spread measured; the bands at each ratio do not overlap. (b) The decisive contrast, in which the routed branch is identical across all four runs.

overlap (Fig. 4a): shared shares of 0.72–0.81 at ratio 0.25, 0.63–0.70 at ratio 0.5, 0.47–0.56 at ratio 1.0 and 0.25–0.37 at ratio 2.0. The ratio-0.5 band is reached from both sides, by narrowing the routed experts at $E = 16$ and by widening the shared one at $E = 8$, and lies 0.07 above the ratio-1.0 band, more than twice the seed spread. Its upper end comes within 0.02 of the ratio-0.25 band, a margin the seed spread does not resolve; the two are told apart only at fixed expert count, where the four $E = 8$ ratios order cleanly at 0.28, 0.47, 0.70 and 0.81. The ordering survives an eightfold range in ratio, an eightfold range in expert count and a sevenfold range in parameter count.

Expert count neither explains the residual spread within a band nor acts consistently: at ratio 2.0 more experts raise the share (0.25 at $E = 4$ to 0.37 at $E = 8$), at ratio 0.25 they lower it (0.79 at $E = 8$ to 0.72 at $E = 32$). Nor is this a capacity confound: holding $E \cdot I = 8192$ equalizes parameter counts, and perplexity is then flat across $E \in \{4, 8, 16, 32\}$ at 2.99, 3.00, 3.00 and 3.02 while the share tracks the ratio that manipulation necessarily varies. Expert count is close to a free parameter for perplexity, and moves the split only within a band's residual spread.

TABLE IV
ROUTING GRANULARITY AT FIXED WIDTHS

| $k$ | $\Delta_{\text{sh}}$ | $\Delta_{\text{rt}}$ | $s_{\min}$ | $s$ | PPL | routed NMI |
|---|---|---|---|---|---|---|
| 1 | +8.89 | +0.29 | 0.97 | 0.97 | 3.13 | 0.0003 |
| 2 | +2.45 | +2.48 | 0.47 | 0.50 | 3.00 | 0.0174 |
| 4 | +0.95 | +3.95 | 0.17 | 0.19 | 2.99 | 0.0235 |

Widths held at $I$=$I_{\text{shared}}$=1024 with $E$=8; two seeds per condition, four at $k$=2. All columns but $s_{\min}$ at 983M tokens.

### A. *Effect of routing granularity*

Because a token passes through $k$ routed experts but only one shared expert, $k$ has an obvious claim to matter. We test it by holding the widths at $I = I_{\text{shared}} = 1024$, so that the width ratio is 1.0 throughout, and varying only $k$.

Table IV shows that $k$ enters, and that it is the strongest single lever in this study: moving from top-1 to top-4 at identical widths swings the split from 0.97 to 0.17, further than the 0.53 obtained by an eightfold change in the shared expert's width. The width-ratio result must therefore be read as holding at fixed routing. This refines rather than refutes the $E/k$ prediction of Section V: it fails on the $E$ axis, where varying $E$ from 4 to 32 at a fixed width ratio leaves the split within the single band 0.47–0.56 and not even in a consistent direction, but succeeds on the $k$ axis and in the predicted direction. What fails is the *ratio* $k/E$; what survives is $k$ alone. Counted per branch rather than per parameter, the routed branch receives $k$ expert passes per token whatever $E$, against one for the shared branch, which matches the direction observed; weighting those passes by width, however, yields the active-width share $\phi$ introduced below, which two runs refute, so the count organizes the $k$ axis without becoming a law.

Unlike the width ratio, this lever incurs a cost. Top-1 costs 0.13 in perplexity and top-4 doubles the routed floating-point cost per token, whereas the width manipulations moved the split across 0.25 to 0.81 within a 0.07 band of perplexity.

A natural repair would fold $k$ into a single scalar, the shared branch's share of *active* feed-forward width $\phi = I_{\text{shared}}/(I_{\text{shared}} + kI)$. Two runs refute it. Both hold $k \cdot I = 2048$ and $I_{\text{shared}} = 1024$, giving the same $\phi = 0.33$ as the reference ($s_{\min} = 0.47$), and vary only how that width is delivered, as four experts of 512 or one of 2048. Both should land near 0.47; neither does. Four-of-512 reaches 0.29 (perplexity 3.06 at 96M parameters), so granularity acts on its own even among $k \geq 2$ configurations: the same active width delivered through more, narrower experts engages the routed branch more. At matched active width and a fixed expert pool this is a controlled test of DeepSeekMoE's fine-grained segmentation argument [1], and it comes out in that argument's favour, on engagement here and on specialization in Section VII. One-of-2048 stays at 0.95, so top-1's collapse is not a width effect either; doubling the per-token routed width changes nothing when the router cannot mix experts. This behaviour is expected: unable to blend experts, the routed branch never becomes functionally useful, its ablation costs

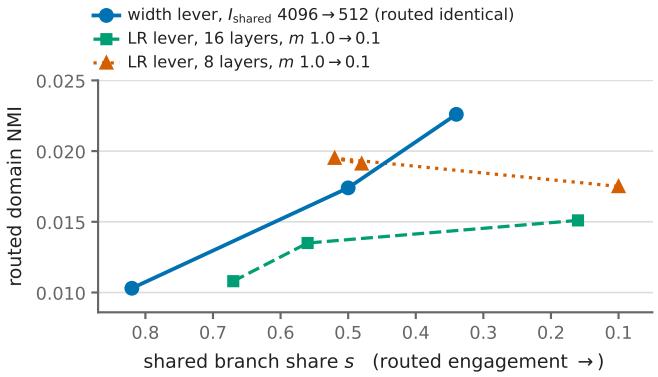



Fig. 5. Routed domain specialization against branch split, engagement increasing to the right; the width series is read at 983M tokens, the learning-rate series at 491M.

0.29 perplexity and its domain NMI *declines* toward zero over training, consistent with the known difficulty of single-expert routing [6].

The data therefore support a joint statement, not a scalar law: at fixed granularity the width proportion orders every configuration we measured; at fixed active width, finer granularity lowers the shared share further; and single-expert routing never engages the routed branch at any width we tested. These results compose with the mechanism. The gradient race decides which branch arrives first, the branch proportion and routing granularity where the allocation settles once it is over. The width manipulations are reparametrizations rather than levers on quality, moving the shared share from 0.25 to 0.81 across the eight 127M–234M configurations at perplexity 2.95 to 3.02.

## VII. Reallocation and Routed Specialization

The motivation for wanting routed experts engaged is usually that they are the object of study, so it matters whether experts forced to carry more computation also organize their work more sharply. We measure gate-weighted normalized mutual information between expert assignment and the domain label of each token, comparing at matched training steps and, where possible, at identical routed architecture (Fig. 5). Shuffling the domain labels over the same tokens gives a permutation null below $10^{-4}$ wherever we computed it, so the values below, small in absolute terms, sit two orders of magnitude above chance, top-1 excepted. The result is not uniform across the two levers.

The width lever is effective. Holding $E = 8$ and $I = 1024$ so that the routed branch is identical across runs and varying only $I_{\text{shared}}$, domain NMI at 983M tokens rises from 0.0103 to 0.0174 to 0.0226 as the shared share falls from 0.82 to 0.50 to 0.34 (values at 983M tokens, not the trajectory minima of Table III): a factor of 2.2 in specialization obtained without touching the routed experts at all, and without cost in perplexity. The granularity runs point the same way, their routed NMI rising from 0.0003 at top-1 to 0.0235 at top-4, and to 0.0301, the highest measured anywhere in this study, when the same active width is delivered as four experts of width 512. Nor is the gain specific to the domain axis: across the same width trio, surface-form NMI rises from 0.118 to 0.141 and frequency NMI from 0.039 to 0.056, with position at the control floor throughout, though the headline factor of 2.2 is domain-specific and the surface-form gain a more modest 1.2. In absolute terms surface form, four lexical classes against three domains, carries six to twelve times more routing information than domain: the experts organize first by form, as the observational literature reports [2], [8] and as a geometric account of routing predicts [12]. The data-centric claim is accordingly not that the routed experts are domain experts, but that how much domain structure they carry is set by the design. Deleting the shared expert makes the same point from the other side: routed domain NMI falls to 0.0113 without it, so the always-on branch, whose absence costs only 0.03 perplexity when the model is trained without it, increases rather than decreases the organization of the routed experts.

The learning-rate lever does not, or not reliably. At 16 layers, measured at 491M tokens where the baseline remains shared-dominated, slowing the shared expert raises NMI monotonically, 0.0108 to 0.0135 to 0.0151, all three points single runs from the same sweep. At 8 layers, where balance has already been reached by then, the response is non-monotone, 0.0191, 0.0195 and 0.0175 for $m = 1.0$, 0.3 and 0.1, ending below the baseline; this series reads the long-schedule reference against short-schedule sweep runs, so we conclude from it only that the gain is not reliable. The dose-response series is similarly equivocal: NMI increases with handicap duration among the ramped runs, but the unhandicapped baseline falls inside their range rather than below it.

What these disagreements share is that specialization tracks how much the routed branch has genuinely been trained, which rises as it takes over computation but flattens near balance and can fall once the shared branch is starved past it. Engagement and specialization are therefore related but distinct axes, and the distinction is practical: experiments requiring more interpretable routed experts should vary the branch widths rather than the learning rate, even though both move the split by comparable amounts. We exclude cross-$E$ comparisons because NMI grows mechanically with expert count, and every value is measured at balancing coefficient $10^{-2}$, which suppresses domain NMI, plausibly by the suppression of shared hidden-state directions that Wang et al. attribute to the balancing loss [12], so magnitudes are conditional on that setting though each contrast is internal to it.

## VIII. Discussion and Conclusion

A small MoE's division of feed-forward labour is only weakly determined by the task. Two quantities a practitioner sets freely, the branches' relative learning rate and relative width, move the split across almost its whole attainable range while leaving perplexity within 0.1; a third, routing granularity, moves it further at a cost the first two do not carry. The shared expert does not perform computation the routed experts could

not; it performs the computation it acquired first, and retains as much of it as its width allows.

Three consequences follow. The first is diagnostic: expert redundancy cannot be read off a single early checkpoint, since 98M and 983M tokens disagree and deeper models stay misleading longer, so branch importance belongs in a trajectory. The second concerns design: our own early sweep over expert count read as a dependence on $E$ only because capacity and parameter count rode along with it, and sweeps should fix what they are not testing. The third is prescriptive. A modest learning-rate handicap engages the routed branch much sooner at no measured cost and need only be applied early, but engagement bought that way does not reliably come with sharper specialization: if the routed experts are wanted merely *used*, either lever serves; if they are wanted *legible*, the width ratio is the one that works. The mechanism also predicts for routing families we did not test, since expert-choice routing [24] equalizes token counts across routed experts but leaves the shared expert's every-token exposure intact, so the race and its remedy should survive it.

For data-centric practice the consequence is concrete. Expert routing is increasingly used as a handle on the training corpus itself: attributing behaviour to data domains, pruning experts that serve rare slices, or editing a model by replacing the experts associated with a subset of the data. Each use assumes that routed experts carry data-domain structure, and that the assumption can be checked by ablation. Our measurements show that the check is valid only once the branches have equalized, and that the amount of domain structure the routed experts carry is itself a design variable: the same corpus, the same token budget and the same routed parameters give a 2.2-fold range in expert–domain mutual information according to the width ratio alone. A pipeline that treats expert–data alignment as a fixed property of a trained model will misattribute in proportion to how early it looks and how the branches were sized.

Several limits bound these claims, and concern generality rather than internal validity. Every model is a DeepSeek-style MoE with a single shared expert, between 58M and 436M parameters, trained for at most 983M tokens on one data mixture and one hardware configuration. What is established is the internal account at that scale; whether the crossover, the controls and the width ordering transfer to models of billions of parameters, to other shared-expert layouts and to other corpora is open, and is the first thing we intend to test. The width-ratio result is an ordering over an eightfold range at one depth, not a functional form, and no single scalar we tried subsumes width and routing at once. The dose-response is replicated at one endpoint, its other points resting on a single seed each. Finally, our evidence is perplexity, ablation impact and routing statistics: at this scale standard benchmarks are near chance, so whether a reallocated split changes downstream behaviour, reasoning or the quality of generated answers, which is what a practitioner would ultimately want, needs a larger model to answer.

Routed experts in small Mixture-of-Experts models are therefore not redundant in the ablation sense: they are outcompeted early by a shared expert that receives gradient from every token. At the scale we studied, where such a model puts its feed-forward computation is, to a good approximation, something the designer chooses rather than something the model discovers.

## Reproducibility

The model, the training code and the analysis toolkit, including whole-branch ablation and routing capture, are available at https://github.com/rcouturier/Tiny-MoE with the run configurations and the per-checkpoint measurements behind every table and figure here; every experiment runs on a single GPU. Two defects found and fixed during the study, a missing causal mask in the absorbed attention inference path and a broken CUDA reduction in the load-balancing loss, are recorded in the accompanying version history, and every measurement reported here postdates both fixes.

## Acknowledgment

We thank Abdelrhman Ebied for releasing Tiny-MoE [18], on which every model in this study is built.

## References

[1] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1280–1297.

[2] N. Muennighoff, L. Soldaini, D. Groeneveld *et al.*, "OLMoE: Open mixture-of-experts language models," in *International Conference on Learning Representations (ICLR)*, 2025.

[3] X. Lu, Q. Liu, Y. Xu *et al.*, "Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 6159–6172.

[4] N. Shazeer, A. Mirhoseini, K. Maziarz *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[5] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[6] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research (JMLR)*, vol. 23, no. 120, pp. 1–39, 2022.

[7] B. Zoph, I. Bello, S. Kumar *et al.*, "ST-MoE: Designing stable and transferable sparse expert models," *arXiv preprint arXiv:2202.08906*, 2022.

[8] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[9] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[10] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[11] A. Yang, B. Yang, B. Hui *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.

[12] X. Wang, S. Hayou, and E. Nalisnick, "The myth of expert specialization in MoEs: Why routing reflects geometry, not necessarily domain expertise," *arXiv preprint arXiv:2604.09780*, 2026.

[13] C. Mouzouni, "Three phases of expert routing: How load balance evolves during mixture-of-experts training," *arXiv preprint arXiv:2604.04230*, 2026.

[14] A. Achille, M. Rovere, and S. Soatto, "Critical learning periods in deep networks," in *International Conference on Learning Representations (ICLR)*, 2019.

[15] J. Frankle, D. J. Schwab, and A. S. Morcos, "The early phase of neural network training," in *International Conference on Learning Representations (ICLR)*, 2020.

[16] D. Dai, L. Dong, S. Ma *et al.*, "StableMoE: Stable routing strategy for mixture of experts," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 7085–7095.

[17] Z. Chen, Y. Deng, Y. Wu *et al.*, "Towards understanding the mixture-of-experts layer in deep learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[18] A. Ebied, "Tiny-MoE: A lightweight mixture-of-experts language model in native PyTorch," https://github.com/AbdelrhmanEbied/Tiny-MoE, 2026.

[19] J. Su, M. Ahmed, Y. Lu *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[20] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[21] G. Penedo, H. Kydlíček, L. Ben Allal *et al.*, "The FineWeb datasets: Decanting the web for the finest text data at scale," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[22] L. Ben Allal, A. Lozhkov, and D. van Strien, "Cosmopedia: How to create large-scale synthetic data for pre-training," Hugging Face Blog, 2024.

[23] K. Paster, M. Dos Santos, Z. Azerbayev, and J. Ba, "OpenWebMath: An open dataset of high-quality mathematical web text," in *International Conference on Learning Representations (ICLR)*, 2024.

[24] Y. Zhou, T. Lei, H. Liu *et al.*, "Mixture-of-experts with expert choice routing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.